SMART INDIA HACKATHON 2026

SARATHI

A local-first LoRA orchestration system

Your private AI. Your machine. Zero token bills.

Problem Statement ID	________________
Problem Statement Title	________________
Theme	**Artificial Intelligence (AI)**
PS Category	**Software**
Team ID	________________
Team Name (Registered on portal)	**Sankalp**

Team Leader: Parag Yewale · Members: Nihar Bute · Srushti Karande · Vedika Bodke · Daksh Somnathe · Shreyash Patil

IDEA TITLE

One AI brain. A team of specialists inside it.

Proposed Solution

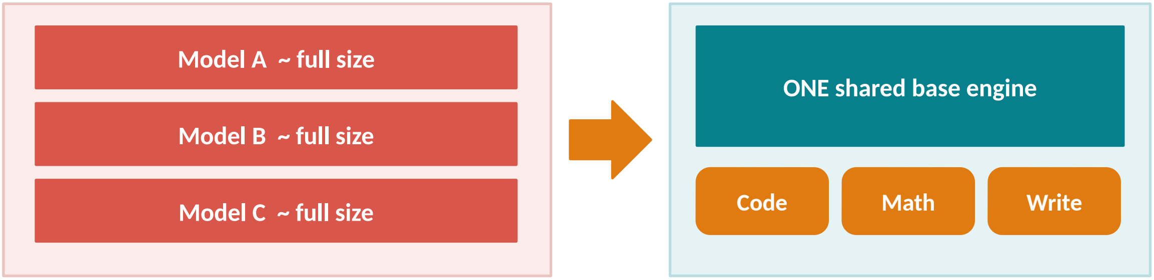
The Swiss-Army-Knife trick

- Keeps ONE small AI engine in memory — not ten heavy ones
- Snaps on a tiny “skill cap” for the job: coding, maths, writing
- Swaps caps live — no restart, no memory flush, no freeze
- Remembers your project across every swap

Today vs. SARATHI

Standard multi-model setup

SARATHI orchestration



VRAM overflow → system freeze

tiny hot-swappable skill caps

How it fixes the three pains

Token Tax	Everyday tasks run free, on your own PC
GPU Memory Wall	One base engine + featherweight caps
Context Loss	Hybrid memory re-injects history on swap

~78%
less system VRAM used

~99%
smaller skill caps vs full model

0 bytes
of your data leave the device

What makes it new

- Auto hardware profiler picks the right-sized free model
- Live cap-swapping without flushing GPU memory
- Memory survives the swap
- Speaks MCP — plugs into the tools you already run

TECHNICAL APPROACH

Four stages, every single prompt

1

Hardware Profiler

Reads your GPU VRAM, RAM & OS

→ Safe base-model pick

2

Intent Analyzer

Reads the prompt, tags the skill needed

→ Target skill tag

3

LoRA Registry & Switcher

Fetches and hot-swaps the skill cap

→ Active model state

4

Hybrid Memory Cache

Re-attaches project history & context

→ Contextual answer

Runs silently in the background as a local server on your own machine — exposed on a loopback address, so nothing ever hits the internet during use.

Technologies to be used

Inference engines

- llama.cpp
- Ollama
- GGUF models

Core language & API

- Python 3.10+
- OpenAI-compatible API
- Loopback socket drivers

Adaptation layer

- LoRA / low-rank adapters
- Adapter registry
- Hugging Face

Memory & connectivity

- Letta
- Mem0
- Zep
- Model Context Protocol (MCP)

Adapter fine-tuning is performed externally — SARATHI only orchestrates and serves.

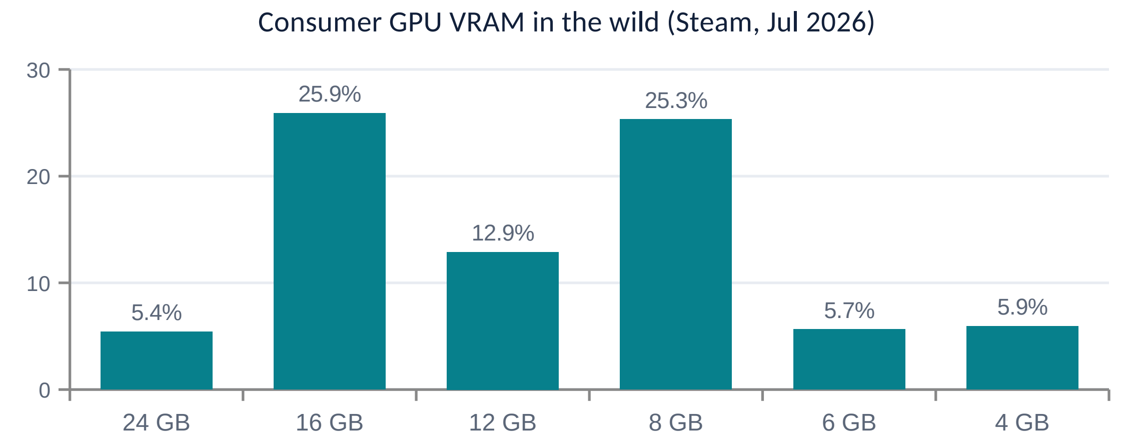
FEASIBILITY AND VIABILITY

Already proven once. Runs on hardware people own.

Working proof of concept
A compact base model (Qwen3 4B) already orchestrated with externally fine-tuned adapters — validated.

Mature open-source base
Built on llama.cpp and Ollama, which already support request-level matrix modification.

Modular team split
Profiler, Classifier, Memory Cache and Adapter Registry build and test independently.



Half of all gaming PCs now have 10 GB+ VRAM — but still not enough for several full models at once. That gap is exactly SARATHI's market.

Potential challenges → how we beat them

Risk	Mitigation strategy
Running out of GPU memory	Hardware profiler caps the base model to what your PC can actually hold
A skill cap fails to load	Runtime validation, error boundaries and graceful fallback to the base engine
Unreliable community models	A certification pipeline separates tested base models from experimental ones
Wrong skill picked for a prompt	Fast intent classifier with per-model runtime profiles (chat template, stop tokens)

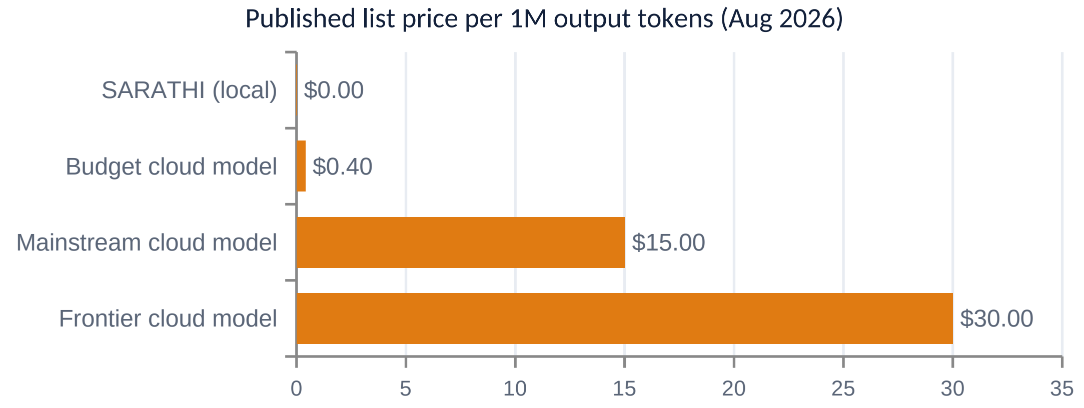
IMPACT AND BENEFITS

Stop renting intelligence. Start owning it.

Students & solo developers
Full AI tooling with no monthly bill

Small firms & startups
Client code and data never leave the office

Regulated & rural users
Works offline; nothing crosses a border



Every routine task you keep off the cloud is a recurring bill that never starts.

Social

- AI access without a credit card or a subscription
- Works where the internet does not — offline by design
- Keeps personal and medical data on the user's own device

Economic

- Removes per-token fees for everyday tasks
- Reuses hardware students and firms already own
- ~99% smaller skill files — cheap to store and share

Environmental

- Small local model instead of a data-centre call for simple work
- ~78% less VRAM → less power drawn per task
- Extends the useful life of existing PCs

78% less VRAM used

99% smaller skill files

0 bytes sent to a server

0 rupees per token

RESEARCH AND REFERENCES

The science this is built on

1. **Hu et al. — LoRA: Low-Rank Adaptation of LLMs, ICLR 2022**
   Specialising a model needs ~0.01% of its parameters — proves the tiny "skill cap" is viable.
   arxiv.org/abs/2106.09685

2. **Chen et al. — Punica: Multi-Tenant LoRA Serving, arXiv:2310.18547**
   Many different skills can be served concurrently over one shared base model.
   arxiv.org/abs/2310.18547

3. **Sheng et al. — S-LoRA: Serving Thousands of Concurrent LoRA Adapters, MLSys 2024**
   Paging adapters in just-in-time serves ~2,000 adapters at near-constant throughput; up to 4× the throughput of vLLM.
   arxiv.org/abs/2311.03285

4. **Gerganov et al. — llama.cpp inference engine**
   Dependency-free C/C++ engine that makes quantised local execution practical on ordinary PCs.
   github.com/ggml-org/llama.cpp

5. **Addair et al. — LoRAX: Multi-LoRA Inference Server**
   Production evidence that dynamic adapter loading works as a served product.
   github.com/predibase/lorax

Where our numbers come from

- 78% VRAM saving, 99% smaller adapters — SARATHI project synopsis
- GPU VRAM share — Steam Hardware Survey, July 2026
- Token prices — published 2026 LLM API list prices (CloudZero comparison)

Standards & ecosystem

- Model Context Protocol — open tool-connection standard, now under the Linux Foundation
- Ollama, Hugging Face, Letta, Mem0, Zep — provider ecosystem

Guided by Prof. Rupali Zambre
Pimpri Chinchwad University, Pune — CSE-AIML

Prototype validated on a compact base model with externally fine-tuned adapters.